

# Звіт

до лабораторної роботи №8

Об'єктно-орієнтоване програмування. Java

Файлове введення-виведення, серіалізація та винятки в Java

Студент: Ісаєв Захар Ігорович

Група: КН-924В

Викладач: Володимир Миколайович Савченко

Харків - 2026

## **Мета роботи**

Мета роботи полягає у формуванні практичних навичок роботи з файловим введенням і виведенням у Java, текстовим і бінарним поданням даних, серіалізацією об'єктів, а також обробкою винятків під час читання, запису, перевірки та відновлення даних.

## **Короткий опис побудованої предметної моделі**

У програмі побудовано модель інформаційних сервісів кампусу. Основними класами предметної області є:

- ExamReminder — нагадування про іспит;
- RoomBooking — бронювання аудиторії;
- ConsultationSlot — слот консультації з викладачем.

Також було створено абстрактний базовий клас CampusService. У ньому винесено спільні поля для всіх сервісів:

- id — ідентифікатор запису;
- studentName — ім'я студента;
- date — дата події;
- time — час події.

Клас ExamReminder містить додаткові поля courseName і auditorium, які описують назву дисципліни та аудиторію проведення іспиту.

Клас RoomBooking містить поля roomNumber і purpose, які описують номер аудиторії та мету бронювання.

Клас ConsultationSlot містить поля teacherName і subject, які описують ім'я викладача та тему консультації.

Таке рішення дозволяє не дублювати однакові поля в кожному класі, а спільну частину винести в один базовий клас.

## **Пояснення ролі класу-контейнера**

Роль класу-контейнера виконує клас CampusServiceRegistry.

Цей клас зберігає список усіх сервісів кампусу в пам'яті програми. Для цього використовується колекція List<CampusService>. Через те, що всі конкретні сервіси наслідуються від CampusService, в одному списку можна

зберігати різні типи об'єктів: ExamReminder, RoomBooking і ConsultationSlot.

У класі CampusServiceRegistry реалізовано такі основні можливості:

- додавання нового сервісу через метод `addService()`;
- перегляд усіх сервісів через метод `getServices()`;
- пошук сервісу за ідентифікатором через метод `findById()`;
- пошук сервісів за іменем студента через метод `findByStudentName()`;
- пошук сервісів за датою через метод `findByDate()`;
- отримання кількості записів через метод `size()`.

Також у контейнері виконується перевірка на дублювання ідентифікатора та конфлікт записів. Наприклад, якщо один студент уже має сервіс на певну дату і час, програма не дозволить додати інший сервіс на той самий момент.

### **Опис текстового формату збереження даних**

Для збереження даних у текстовий файл використовується простий формат, у якому кожен об'єкт записується в один рядок. Поля розділяються символом `;`.

Загальний формат рядка:

`TYPE;id;studentName;date;time;field1;field2`

Поле `TYPE` визначає тип сервісу. У програмі використовуються такі типи:

`EXAM`

`ROOM`

`CONSULTATION`

Формат для нагадування про іспит:

`EXAM;id;studentName;date;time;courseName;auditorium`

Приклад:

`EXAM;E001;Zakhar Isaiev;2026-05-20;10:30;Object-Oriented Programming;301`

Формат для бронювання аудиторії:

ROOM;id;studentName;date;time;roomNumber;purpose

Приклад:

ROOM;R001;Zakhar Isaiev;2026-05-21;14:00;204;Team project meeting

Формат для консультації:

CONSULTATION;id;studentName;date;time;teacherName;subject

Приклад:

CONSULTATION;C001;Zakhar Isaiev;2026-05-22;12:15;Savchenko  
V.;Java serialization

Обов'язковими полями для всіх записів є:

- тип сервісу;
- ідентифікатор;
- ім'я студента;
- дата;
- час;
- два додаткові поля залежно від типу сервісу.

Під час читання з текстового файлу програма перевіряє, щоб у кожному рядку було 7 полів. Якщо кількість полів неправильна або тип сервісу невідомий, створюється виняткова ситуація.

### **Пояснення, як виконано серіалізацію і десеріалізацію**

Серіалізацію і десеріалізацію реалізовано в окремому класі CampusServiceFileManager.

Серіалізація виконується методом `serialize()`. У цьому методі об'єкт CampusServiceRegistry записується у бінарний файл за допомогою `ObjectOutputStream`.

Десеріалізація виконується методом `deserialize()`. У цьому методі дані з бінарного файлу читаються за допомогою `ObjectInputStream`, після чого відновлюється об'єкт CampusServiceRegistry.

Для можливості серіалізації класи моделі реалізують інтерфейс `Serializable`. Це потрібно для того, щоб Java могла зберегти стан об'єктів у вигляді послідовності байтів.

У роботі серіалізація використовується окремо від текстового формату. Текстовий файл потрібний для зручного перегляду та ручної перевірки даних, а бінарний файл потрібний для швидкого збереження і відновлення всього стану реєстру.

### **Опис винятків, які можуть виникати під час роботи програми**

Під час роботи програми можуть виникати такі винятки:

`IOException` — виникає при помилках роботи з файлами. Наприклад, якщо файл не можна створити, прочитати або записати.

`ClassNotFoundException` — може виникнути під час десеріалізації, якщо у бінарному файлі збережено об'єкт класу, який програма не може знайти.

`InvalidCampusServiceException` — власний користувацький виняток предметної області. Він використовується для помилок, пов'язаних із сервісами кампусу. Наприклад, якщо сервіс має дубльований ідентифікатор або студент уже має інший сервіс у той самий час.

`IllegalArgumentException` — може виникати при створенні об'єктів з неправильними даними. Наприклад, якщо передано порожній `id`, порожнє ім'я студента, відсутню дату або час.

Також помилки можуть виникати при читанні текстового файлу, якщо рядок має неправильну кількість полів, невідомий тип сервісу або неправильний формат дати чи часу.

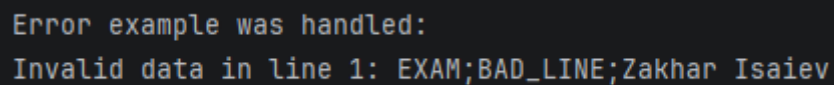
### **Приклад помилкової ситуації та спосіб її обробки**

У програмі в класі `Main` показано приклад помилкової ситуації. Для цього створюється неправильний текстовий файл з таким рядком:

```
EXAM;BAD_LINE;Zakhar Isaiev
```

Цей рядок є неправильним, тому що в ньому тільки 3 поля замість 7. Метод `loadFromText()` намагається прочитати цей рядок, але під час перевірки формату виникає помилка.

Помилка перехоплюється за допомогою блоку try-catch. Програма не завершується аварійно, а виводить зрозуміле повідомлення:



```
Error example was handled:
Invalid data in line 1: EXAM;BAD_LINE;Zakhar Isaiev
```

Рис. 1 - Повідомлення про помилкову ситуацію  
**Додаткове завдання**

Для додаткового завдання обрав модель з попередньої лабораторної роботи №7. У тій роботі розглядалася предметна область інформаційних сервісів кампусу. Основними класами моделі були ExamReminder, RoomBooking і ConsultationSlot. Також використовувалися інтерфейси Notifiable і Schedulable, які описували можливість повідомлення студента та планування події.

Цю модель можна адаптувати до зовнішнього збереження даних, тому що її об'єкти мають прості поля, які можна записати у текстовий файл. Для цього доцільно використати формат, у якому кожен об'єкт зберігається в окремому рядку, а поля розділяються символом крапка з комою.

Запропонований текстовий формат збереження:

EXAM;id;studentName;date;time;courseName;auditorium

ROOM;id;studentName;date;time;roomNumber;purpose

CONSULTATION;id;studentName;date;time;teacherName;subject

Наприклад:

EXAM;E001;Zakhar Isaiev;2026-05-20;10:30;Object-Oriented  
Programming;301

ROOM;R001;Zakhar Isaiev;2026-05-21;14:00;204;Team project meeting

CONSULTATION;C001;Zakhar Isaiev;2026-05-22;12:15;Savchenko  
V.;Java serialization

Обов'язковими полями для всіх записів є тип сервісу, ідентифікатор, ім'я студента, дата та час. Для ExamReminder додатково обов'язковими є назва

дисципліни та аудиторія. Для RoomBooking обов'язковими є номер аудиторії та мета бронювання. Для ConsultationSlot обов'язковими є ім'я викладача та тема консультації.

Можливими помилками формату є неправильна кількість полів у рядку, порожній ідентифікатор, відсутнє ім'я студента, неправильний формат дати або часу, невідомий тип запису, а також конфліктні записи, коли один студент має дві події на одну дату й один час.

Серіалізація всієї колекції об'єктів для цієї моделі є доцільною. Вона дозволяє швидко зберегти повний стан реєстру сервісів кампусу у бінарний файл і потім відновити його без ручного розбору кожного поля. Але серіалізація не повинна повністю замінювати текстовий формат, тому що текстовий файл зручніший для перегляду, перевірки та ручного редагування.

### Короткий висновок щодо додаткового завдання

У результаті аналізу було визначено, що модель з лабораторної роботи №7 можна адаптувати до зовнішнього збереження даних. Для цього достатньо визначити чіткий текстовий формат, перевіряти обов'язкові поля та обробляти можливі помилки формату. Серіалізація всієї колекції також є доцільною, оскільки вона дозволяє зберегти і відновити повний стан реєстру сервісів кампусу.

### Робота програми та UML

```
Initial campus services:
ExamReminder {id='E001', studentName='Zakhar Isaiev', date=2026-05-20, time=10:30, courseName='Object-Oriented Programming', auditorium='301'}
RoomBooking {id='R001', studentName='Zakhar Isaiev', date=2026-05-21, time=14:00, roomNumber='204', purpose='Team project meeting'}
ConsultationSlot {id='C001', studentName='Zakhar Isaiev', date=2026-05-22, time=12:15, teacherName='Valentina V.', subject='Java serialization'}

Restored from text file:
ExamReminder {id='E001', studentName='Zakhar Isaiev', date=2026-05-20, time=10:30, courseName='Object-Oriented Programming', auditorium='301'}
RoomBooking {id='R001', studentName='Zakhar Isaiev', date=2026-05-21, time=14:00, roomNumber='204', purpose='Team project meeting'}
ConsultationSlot {id='C001', studentName='Zakhar Isaiev', date=2026-05-22, time=12:15, teacherName='Valentina V.', subject='Java serialization'}

Restored from binary file:
ExamReminder {id='E001', studentName='Zakhar Isaiev', date=2026-05-20, time=10:30, courseName='Object-Oriented Programming', auditorium='301'}
RoomBooking {id='R001', studentName='Zakhar Isaiev', date=2026-05-21, time=14:00, roomNumber='204', purpose='Team project meeting'}
ConsultationSlot {id='C001', studentName='Zakhar Isaiev', date=2026-05-22, time=12:15, teacherName='Valentina V.', subject='Java serialization'}

Search by student name:
ExamReminder {id='E001', studentName='Zakhar Isaiev', date=2026-05-20, time=10:30, courseName='Object-Oriented Programming', auditorium='301'}
RoomBooking {id='R001', studentName='Zakhar Isaiev', date=2026-05-21, time=14:00, roomNumber='204', purpose='Team project meeting'}
ConsultationSlot {id='C001', studentName='Zakhar Isaiev', date=2026-05-22, time=12:15, teacherName='Valentina V.', subject='Java serialization'}

Error example was handled:
Invalid data in line 1: EXAM;BAD_LINE;Zakhar Isaiev
```

Рис. 1 - Робота програми

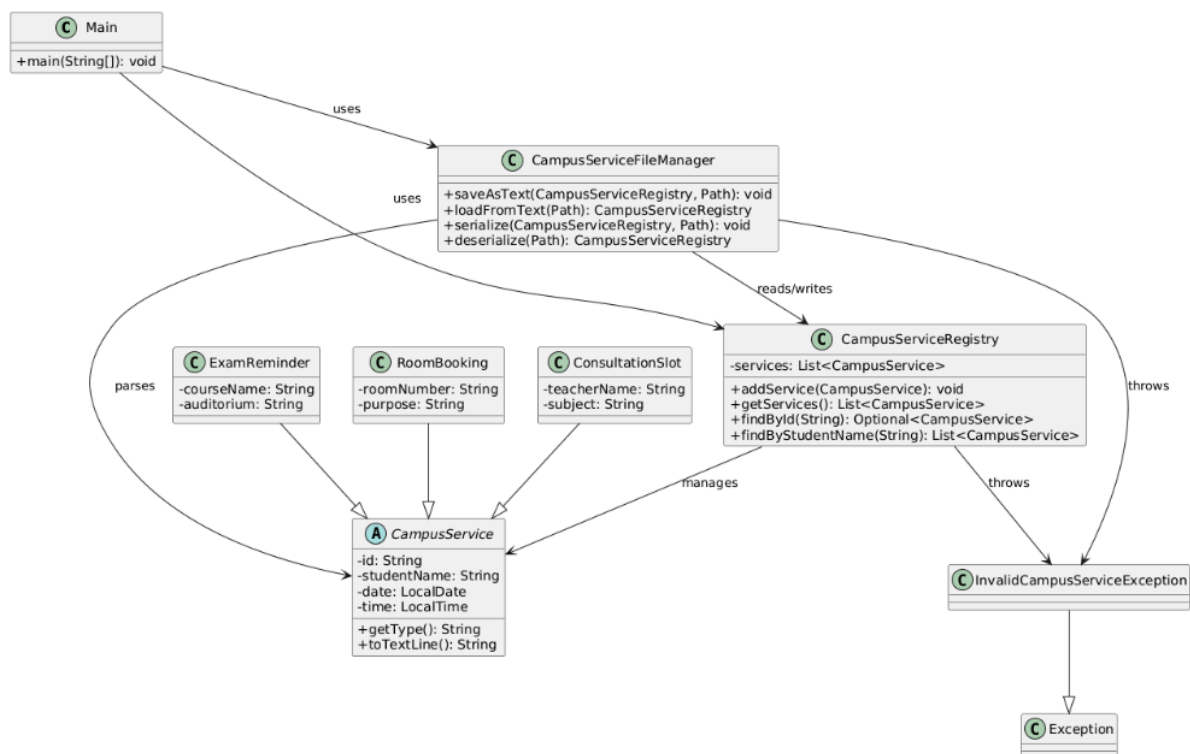


Рис. 2 - UML-діаграма моделі інформаційних сервісів кампусу, реєстру та підсистеми збереження даних

## Висновок

У ході лабораторної роботи №8 було розроблено Java-застосунок для роботи з інформаційними сервісами кампусу. Було створено модель предметної області з класами ExamReminder, RoomBooking і ConsultationSlot, а також базовий клас CampusService для спільних полів. Для зберігання об'єктів у пам'яті використано клас-контейнер CampusServiceRegistry, а файлові операції винесено в окремий клас CampusServiceFileManager. У програмі реалізовано запис даних у текстовий файл, читання з текстового файлу, серіалізацію та десеріалізацію реєстру у бінарний файл. Також було створено власний виняток InvalidCampusServiceException і показано обробку помилкової ситуації під час читання некоректного рядка з файлу. У результаті роботи було закріплено навички файлового введення-виведення, серіалізації об'єктів та обробки винятків у Java.